Assignment: Recurrent Neural Networks (RNN)

Topic: Sequential Modeling, Backpropagation Through Time (BPTT), and Gradient Dynamics

Problem Context

Consider a Recurrent Neural Network (RNN) processing a sequence of three time-step inputs **(x_1, x_2, x_3)**. The hidden state at each time step *t* is updated via the recurrence relation:

$$h_t = f(x_t, h_{t-1})$$

The scalar loss *L* is evaluated at the final step as a function of the terminal hidden state h_3:

$$L = L(h_3)$$

Questions

1. Forward Pass

Explain how the RNN processes the sequence (x_1, x_2, x_3). Draw an unrolled computational graph illustrating the dependency flow among the inputs x_t, hidden states h_t, and the final loss L.

2. Hidden State Dynamics

Explain the functional role of the hidden state h_t. How does h_t accumulate and carry context from preceding time steps across the sequence?

3. Backpropagation Through Time (BPTT)

Explain how the scalar loss L is backpropagated from h_3 to h_1. Why is this optimization process specifically termed Backpropagation Through Time?

4. Chain Rule Derivation

Using the multivariable chain rule, derive the explicit analytical expression for the

gradient: $\partial L / \partial h_1$

Show each step of the expansion through intermediate hidden states h_2 and h_3.

5. Intermediate Activations & Memory

During the forward pass, hidden states (h_1, h_2, h_3) are computed sequentially. Explain why and how these intermediate activation vectors must be stored in memory for use during the backward pass.

6. Parameter Sharing

Explain why the same weight parameters are shared across every time step in an RNN. How does parameter sharing affect gradient computation and parameter updates during BPTT?

7. Vanishing and Exploding Gradients

Explain why gradients can exponentially diminish (vanish) or grow (explode) when BPTT is performed over long temporal sequences.

8. Practical Implementation (PyTorch)

Using PyTorch, implement a simple custom RNN cell or module that processes a sequence of at least 5 time steps ($T \geq 5$). Compute a scalar loss, call .backward(), and print the gradient tensor (.grad) for each model parameter. Explain what these parameter gradients physically represent.

Expected Learning Outcomes

• **Trace and visualize** the forward computation and data flow of an unrolled RNN sequence. • **Conceptualize hidden states** as dynamic temporal memory carrying historical sequence context. • **Derive loss gradients** across time steps using the multivariable chain rule.
• **Understand BPTT mechanics** and the resulting memory overhead during backpropagation. • **Explain parameter sharing** and its impact on gradient accumulation across time steps. • **Diagnose gradient instability** including the mathematical causes of vanishing and exploding gradients.
• **Implement recurrent models** and inspect parameter gradients using PyTorch.